

Medidas de seguridad aplicables al proyecto

1. Alcance

Este documento resume los esquemas de seguridad mas relevantes para el proyecto de tienda web. La arquitectura actual usa un frontend Vue/Vite publicado en GitHub Pages, un backend ASP.NET Core expuesto con Caddy por HTTPS, SQL Server en Docker y despliegue con Docker Compose.

2. Medidas ya presentes

2.1 HTTPS con Caddy

El backend se expone mediante Caddy en el dominio `ci0137-seg-fault.duckdns.org`. Esto permite usar HTTPS y evita que credenciales, datos personales o informacion de checkout viajen en texto claro.

Consideraciones:

- El dominio debe apuntar a la IP publica del servidor.
- Los puertos `80` y `443` deben estar abiertos.
- Caddy debe reenviar las peticiones al backend con `reverse_proxy backend_api:8080`.

2.2 CORS configurable

El backend permite configurar los origenes autorizados por variables de entorno:

```
Cors__AllowedOrigins__0: https://kattyab.github.io
```

Esto evita usar `AllowAnyOrigin` en produccion y permite aceptar solo el frontend publicado. Para GitHub Pages, el origen correcto es `https://kattyab.github.io`; la ruta `/CI0137_seg_fault/` no forma parte del origen CORS.

2.3 Hash de contraseñas

El sistema no guarda contraseñas en texto claro. Usa sal aleatoria y PBKDF2 con SHA-256 para almacenar hashes. Esto reduce el impacto si la base de datos se filtra.

2.4 Validacion de pagos

El checkout valida longitud de tarjeta, algoritmo de Luhn, fecha de expiracion, CVC y BIN permitido. Para seguridad, no se debe guardar el numero completo de tarjeta ni el CVC; solo datos como BIN, ultimos 4 digitos, marca, estado y monto.

3. Medidas recomendadas

3.1 Autenticacion real y autorizacion por endpoint

Actualmente el token devuelto por login es simple y no funciona como una sesion validada en el servidor. La mejora mas importante es implementar JWT o cookies seguras y proteger endpoints con autorizacion.

Recomendacion:

- Usar JWT Bearer para la SPA o cookies **HttpOnly**, **Secure** y **SameSite**.
- Proteger rutas como checkout, historial de ordenes y perfil con **[Authorize]**.
- Obtener el usuario desde el token, no desde parametros manipulables por el cliente.
- Guardar la llave de firma del token como secreto, fuera del repositorio.

3.2 Proteccion contra CSRF

CSRF ocurre cuando un sitio externo intenta provocar acciones en una aplicacion donde el usuario ya esta autenticado. Este riesgo es importante si el proyecto usa cookies de sesion, porque el navegador las envia automaticamente.

Beneficio:

- Evita que un sitio malicioso intente ejecutar acciones como checkout o cambios de cuenta en nombre del usuario.
- Refuerza endpoints que modifican datos, como registro, login, ordenes y pagos.
- Complementa CORS, porque CORS no reemplaza una validacion de intencion en el servidor.

Consideracion:

- Si se usan cookies, configurarlas con **Secure**, **HttpOnly** y **SameSite**.
- Para operaciones sensibles, exigir un token CSRF enviado en un encabezado custom y validado por el backend.

3.3 Usuario SQL con permisos minimos

La API no deberia conectarse a SQL Server con **sa**. Se recomienda crear un usuario especifico para la aplicacion con permisos solo sobre la base **web-development-26**.

Beneficio:

- Si la API se compromete, el atacante no obtiene privilegios administrativos sobre SQL Server.
- Reduce el impacto de errores de aplicacion o inyecciones SQL.

3.4 Encabezados HTTP de seguridad

Caddy puede agregar encabezados para endurecer el comportamiento del navegador:

```
header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    Referrer-Policy "strict-origin-when-cross-origin"
    X-Frame-Options "DENY"
    Permissions-Policy "geolocation=(), microphone=(), camera=()"
}
```

Beneficio:

- Fuerza HTTPS con HSTS.
- Reduce riesgos de clickjacking, sniffing de contenido y fuga de informacion por headers.

Consideracion:

- HSTS debe probarse con cuidado; si el dominio o certificado queda mal configurado, los navegadores pueden bloquear el acceso temporalmente.

3.5 Rate limiting en login y checkout

El login, registro y checkout son endpoints sensibles. Deben tener limite de peticiones por IP, usuario o correo.

Beneficio:

- Reduce fuerza bruta contra contraseñas.
- Reduce intentos automatizados de tarjetas.
- Protege la API contra abuso.

Consideracion:

- ASP.NET Core permite aplicar rate limiting por ruta y responder con **429 Too Many Requests** cuando se excede el limite.